Math reference

Latin American regional preparation. C++17; zero-based indices; array ranges $[l, r)$ unless stated. Choose the model, check bounds, then type. Edition 2026-10-18.

Lookup

1 Math 1
- 1.1 Choose the reduction . . . 1
- 1.2 Exact arithmetic . . . 1
- 1.3 Congruences . . . 1
- 1.4 Prime exponents and counting values . . . 2
- 1.5 Remainder states and decimals . . . 2
- 1.6 Approximation as a construction . . . 3
- 1.7 Combinatorics . . . 3
- 1.8 Probability and expectation . . . 3
- 1.9 Proof and inequality checks . . . 4
- 1.10 Integer geometry . . . 4

Math (1)

1.1 Choose the reduction

Huge integer + allowed error: bound discarded digits before factoring. Many occurrences, values $\le V$: histogram/multiples, not all pairs. Decimal division: reduce the denominator, then remainder states. Sum over subarrays/pairs: count each feature's contribution. Repeated operations: seek an invariant; necessity is not sufficiency. Random process: indicators or expectation recurrence before simulation.

1.2 Exact arithmetic

Prelude.h

Description: Shared includes, aliases and bounds. Include before the modules.

<algorithm>, <cstdint>, <limits>, <map>, <numeric>, <optional>, <stdexcept>, <string>, <tuple>, <utility>, <vector> 895a16, 5 lines


```
namespace finals_math {
using i64 = long long;
using i128 = __int128_t; // GCC/Clang extension supported by
    our C++17 build.

} // namespace finals_math
```

1.2.1 Floor, ceiling and bounds

C++ division truncates toward zero: $-7/3 = -2$, but $\lfloor -7/3 \rfloor = -3$. For $a \ge 0, b > 0$, $\lceil a/b \rceil = a/b + [a \bmod b \neq 0]$; avoids overflow in $a + b - 1$. For positive d, multiples in $[L, R]$ correspond to integer indices $[\lceil L/d \rceil, \lfloor R/d \rfloor]$; answer is zero if empty.

Widen *before* arithmetic: `i128(a)*b`, `i128(b.x)-a.x`. Assignment to a wide result is too late. Signed 64-bit maximum is about $9.22 \cdot 10^{18}$. Bound the whole sum, not only a term: pair counts $O(n^2)$, all subarray sums $O(n^3 A)$ for $|a_i| \le A$. Helpers reject division by zero and an unrepresentable quotient. `i128` requires GCC/Clang; not arbitrary precision.

IntegerDivision.h

Description: Checked narrowing and signed division.

"Prelude.h" 16887e, 20 lines


```
namespace finals_math {
inline i64 narrow(i128 x) {
    if (x < std::numeric_limits<i64>::min() || x > std::
        numeric_limits<i64>::max())
        throw std::overflow_error("signed 64-bit result");
    return static_cast<i64>(x);
}
// b != 0; throws when the quotient cannot fit i64 (LLONG_MIN /
    -1).
inline i64 floor_div(i64 a, i64 b) {
    if (!b) throw std::invalid_argument("zero denominator");
    i128 x=a, y=b, q=x/y, r=x%y;
    if (r && ((r<0)!=(y<0))) --q;
    return narrow(q);
}
inline i64 ceil_div(i64 a, i64 b) {
    if (!b) throw std::invalid_argument("zero denominator");
    i128 x=a, y=b, q=x/y, r=x%y;
    if (r && ((r<0)==(y<0))) ++q;
    return narrow(q);
}
} // namespace finals_math
```

1.2.2 Exact decimal input

For at most k fractional places, store the text scaled by 10^k. Cents: `12.05`$\to 1205$, `3`$\to 300$, `-0.07`$\to -7$. Read a string, not a `double`. Parser rejects excess fractional places; it does not round. No exponent notation or commas. Digit count excludes the sign; zero has one digit. Integer division by ten is exact at powers of ten; `log10` can misclassify a boundary.

ExactDecimals.h

Description: Exact decimals and digit counts.

"Prelude.h", "IntegerDivision.h" cc977c, 32 lines


```
namespace finals_math {
// Exact fixed-point parse: optional sign; digits; optional dot
    and digits.
// Requires at least one digit before '.', <= places fractional
    digits, 0<=places<=18.
inline i64 parse_fixed(const std::string& s, int places) {
    if (places<0 || places>18) throw std::invalid_argument("
        decimal places");
    std::size_t j=0; bool neg=false;
    if (j<s.size() && (s[j]=='+' || s[j]=='-')) neg=s[j++]=='-'
        ;
    i128 value=0; int whole=0, frac=0;
    const i128 cap=i128(std::numeric_limits<i64>::max())+1;
    auto digit=[&](char c) {
        if (c<'0'||c>'9') throw std::invalid_argument("decimal
            digit");
        value=value*10+(c-'0');
        if (value>cap) throw std::overflow_error("decimal value
            ");
    };
    while(j<s.size() && s[j]!='.') { digit(s[j++]); ++whole; }
    if(!whole) throw std::invalid_argument("integer part");
    if(j<s.size()) {
        ++j;
        while(j<s.size()) { digit(s[j++]); ++frac; }
    }
    if(frac>places) throw std::invalid_argument("too many
        fractional digits");
    while(frac++<places) {
        value*=10;
        if(value>cap) throw std::overflow_error("decimal value"
            );
    }
    return narrow(neg ? -value : value);
}
inline int digits10(i64 x) { // Counts magnitude digits,
    including zero, excluding sign.
    i128 v=x; if(v<0) v=-v;
    int d=1; while(v>=10) { v/=10; ++d; } return d;
}
} // namespace finals_math
```

1.3 Congruences

1.3.1 Powers and inverses

Normalize residues into $[0, m)$, $m > 0$. Addition/multiplication preserve congruence; division usually does not. An inverse of a exists iff $\gcd(a, m) = 1$. For prime p and $p \nmid a$, $a^{-1} = a^{p-2} \pmod p$. Composite modulus: use extended Euclid. Example $3^{-1} = 3 \pmod 8$, but $2^{-1} \pmod 8$ does not exist. Binary powering: $O(\log e)$ time, $O(1)$ space, $e \ge 0$. A widened product is needed even when both residues fit 64 bits. Our exponent-zero convention gives $0^0 = 1$ before reduction.

ModularArithmetic.h

Description: Modular arithmetic and binary powers.

"Prelude.h" d4eae5, 13 lines


```
namespace finals_math {
inline i64 norm(i64 a, i64 m) {
    if(m<=0) throw std::invalid_argument("positive modulus
        required");
    a%=m; return a<0 ? a+m : a;
}
inline i64 mul_mod(i64 a, i64 b, i64 m) {
    return i64(i128(norm(a,m))*norm(b,m)%m);
}
inline i64 pow_mod(i64 a, std::uint64_t e, i64 m) {
    a=norm(a,m); i64 r=1%m;
    while(e) { if(e&1) r=mul_mod(r,a,m); a=mul_mod(a,a,m); e
        >>=1; } return r;
}
} // namespace finals_math
```

1.3.2 Bezout and shared-factor CRT

Extended Euclid gives $ax + by = g = \gcd(a, b)$ in $O(\log \min(a, b))$. $ax + by = c$ has integer solutions iff $g \mid c$. From one solution (x_0, y_0), all are $(x_0 + tb/g,\ y_0 - ta/g)$, $t \in \mathbb{Z}$. Use floor/ceiling to intersect bounds on t. If $a = b = 0$, handle $c = 0$ separately.

Merge $X \equiv a \pmod m$, $X \equiv b \pmod n$: $g = \gcd(m,n)$ must divide $b-a$. Put $X = a + mt$ and solve

$$(m/g)t \equiv (b-a)/g \pmod{n/g}.$$

Answer is unique modulo $\operatorname{lcm}(m,n)$. Example $(a,m) = (2,6), (b,n) = (5,9)$ gives $X = 14 \pmod{18}$; residue 4 modulo 9 is incompatible. Merge many equations one by one. Positive moduli; modulus one adds no restriction. Helper throws if the merged lcm exceeds 64 bits.

EuclidCRT.h

Description: Extended Euclid, inverses and CRT.

"Prelude.h", "IntegerDivision.h", "ModularArithmetic.h" 2d59a9, 34 lines

```
namespace finals_math {
// Requires nonnegative a,b. Returns g,x,y with a*x+b*y=g.
inline std::tuple<i64,i128,i128> extended_gcd(i64 a,i64 b) {
    if(a<0||b<0) throw std::invalid_argument("nonnegative gcd
        inputs");
    i128 r=a,s=b,x=1,y=0,u=0,v=1;
    while(s) {
        i128 q=r/s;
        i128 t=r-q*s; r=s; s=t;
        t=x-q*u; x=u; u=t;
        t=y-q*v; y=v; v=t;
    }
    return {i64(r),x,y};
}
inline std::optional<i64> inverse(i64 a,i64 m) {
    if(m<=1) throw std::invalid_argument("inverse modulus must
        exceed one");
    auto [g,x,y]=extended_gcd(norm(a,m),m); (void)y;
    if(g!=1) return std::nullopt;
    return i64((x%m+m)%m);
}
struct Congruence { i64 residue, modulus; };
// Merge x=a (mod m), x=b (mod n); nullopt = incompatible.
// Positive moduli. Throws if the merged lcm exceeds LLONG_MAX.
inline std::optional<Congruence> crt(Congruence a,Congruence b)
     {
    a.residue=norm(a.residue,a.modulus); b.residue=norm(b.
        residue,b.modulus);
    auto [g,x,y]=extended_gcd(a.modulus,b.modulus); (void)y;
    i128 delta=i128(b.residue)-a.residue;
    if(delta%g) return std::nullopt;
    i64 n=b.modulus/g;
    i128 t=(delta/g*x)%n; if(t<0)t+=n;
    i64 lcm=narrow(i128(a.modulus)*n);
    i128 r=(i128(a.residue)+i128(a.modulus)*t)%lcm;
    return Congruence{i64(r),lcm};
}
} // namespace finals_math
```

1.4 Prime exponents and counting values

1.4.1 Factorization and totient

For $n = \prod p^{e_p}$,

$$\tau(n) = \prod (e_p + 1), \quad \sigma(n) = \prod (1 + p + \cdots + p^{e_p}),$$

$$\phi(n) = n \prod_{p \mid n} (1 - 1/p).$$

Example $72 = 2^3 3^2$: $\tau = 12$, $\sigma = 195$, $\phi = 24$. For one, the empty products give all three values one. Generate divisors by extending old divisors with each power of a new prime. Geometric accumulation avoids division modulo a composite modulus.

Trial division stops at $p \le n/p$, reducing n as factors are removed; remaining $n > 1$ is prime. Worst case $O(\sqrt{n})$. A ten-digit core needs candidates only through 10^5; repeated arbitrary 64-bit factorization needs a different algorithm. SPF sieve: $O(V)$ time/space; factor $x \le V$ in $O(\log x)$. Zero/negative inputs require separate definitions.

FactorSieve.h

Description: Trial factorization, totients and sieve.

"Prelude.h" 03ad31, 31 lines

```
namespace finals_math {
// Positive n; O(sqrt(n)) worst case. Suitable for n<=1e12 in
    modest counts.
inline std::vector<std::pair<i64,int>> factor(i64 n) {
    if(n<1) throw std::invalid_argument("positive factor input"
        );
    std::vector<std::pair<i64,int>> f;
    for(i64 p=2;p<=n/p;p+=(p==2?1:2)) if(n%p==0) {
        int e=0; do {n/=p;++e;}while(n%p==0); f.push_back({p,e}
            );
    }
    if(n>1)f.push_back({n,1}); return f;
}
inline i64 phi(i64 n) {
    i64 ans=n; for(auto [p,e]:factor(n)) { (void)e; ans=ans/p*(
        p-1); } return ans;
}
struct Sieve {
    std::vector<int> spf, primes;
    explicit Sieve(int n) {
        if(n<0) throw std::invalid_argument("nonnegative sieve
            limit");
        spf.resize(std::size_t(n)+1,0);
        for(int i=2;i<=n;++i) {
            if(!spf[i]) { spf[i]=i; primes.push_back(i); }
            for(int p:primes) { if(p>spf[i] || p>n/i)break; spf
                [p*i]=p; }
        }
    }
    std::vector<std::pair<i64,int>> factors(int x) const {
        if(x<1 || x>=int(spf.size())) throw std::
            invalid_argument("sieve range");
        std::vector<std::pair<i64,int>> f;
        while(x>1) {int p=spf[x],e=0;do{x/=p;++e;}while(x>1&&
            spf[x]==p);f.push_back({p,e});}
        return f;
    }
};
} // namespace finals_math
```

1.4.2 Histogram and exact gcd classes

For positive values $\le V$, let $f[x]$ count occurrences and $M[d] = \sum_{k \ge 1} f[kd]$. All M take $O(V \log V)$ time, $O(V)$ memory. Zero needs separate handling. For fixed b and divisors $g \mid b$, process decreasing g:

$$E_b[g] = M[g] - \sum_{\substack{h \mid b,\ g \mid h \\ h > g}} E_b[h].$$

Then $E_b[g]$ counts numerators with $\gcd(a,b) = g$ exactly. For values $[2,3,6,6]$, $b = 6$: $E[6] = 2, E[3] = 1, E[2] = 1, E[1] = 0$. For ordered occurrence pairs multiply by $f[b]$. Nested divisor work needs its own bound, e.g. $\sum_b \tau(b)^2$; it is not automatically $O(V \log V)$.

1.5 Remainder states and decimals

1.5.1 Digits as a small state

Append digit d to remainder r modulo m: $r' = (10r + d) \bmod m$. Widen $10r$ if needed. At the same position, equal remainder means the same possible suffixes when no other constraint distinguishes prefixes. Hence a digit-pattern DP uses $(\text{position}, r)$, with $O(10Lm)$ time for length L. Leading-zero rules and extra digit restrictions must enter transitions/state. Smallest reconstruction: suffix feasibility, then try digits in increasing order while retaining a feasible suffix. Counting and existence differ.

1.5.2 Period and rational display

For $\gcd(a,m) = 1$, multiplicative order $\operatorname{ord}_m(a)$ is the smallest $k > 0$ with $a^k \equiv 1 \pmod m$. It divides $\phi(m)$; equality need not hold. Start $k = \phi(m)$; for prime $p \mid k$, repeatedly divide by p when $a^{k/p} \equiv 1 \pmod m$. Factorization dominates large inputs. Example $\operatorname{ord}_9(10) = 1$, while $\phi(9) = 6$.

For $a \ge 0, b > 0$, reduce $q = b/\gcd(a,b) = 2^u 5^v r$, $\gcd(r,10) = 1$. Canonical displayed digit count (no point or overbar):

$$D(a,b) = \operatorname{digits}(\lfloor a/b \rfloor) + \max(u,v) + P(r),$$

where $P(1) = 0$, else $P(r) = \operatorname{ord}_r(10)$. Reason: long division advances remainder by multiplication by ten; after the $2,5$ prefix, a repeated remainder requires $10^k \equiv 1$. Do not count artificial trailing zeros or the alternative recurring-nine form.

Examples: $1/4$: $1+2+0 = 3$; $1/6$: $1+1+1 = 3$; $3/14$: $1+1+6 = 8$; $41/4$: $2+2+0 = 4$. Zero has one integer digit. *Displaying Decimals (2025 D):* separate fractional length by gcd classes; integer digits add one per threshold $a \ge 10b, 100b, \ldots$, answered with frequency suffix counts. For $[15,36,14]$, nine ordered lengths sum to 30; expectation $10/3$. The single-fraction formula passed direct long-division checks; full maximum-size aggregation is not benchmarked here.

DecimalPeriod.h
Description: Multiplicative order and decimal shape.
"Prelude.h", "ExactDecimals.h", "ModularArithmetic.h", "FactorSieve.h" 8056f1, 16 lines

```
namespace finals_math {
inline i64 order_mod(i64 a,i64 m) { // m>=2, gcd(a,m)=1; trial division cost applies.
    if(m<2 || std::gcd(norm(a,m),m)!=1) throw std::invalid_argument("order requires a unit");
    i64 k=phi(m);
    for(auto [p,e]:factor(k)) { (void)e; while(k%p==0&&pow_mod(a,k/p,m)==1)k/=p; }
    return k;
}
struct DecimalShape { int integer_digits, nonrepeating; i64 period; };
// a>=0, b>=1; period computation may be costly.
inline DecimalShape decimal_shape(i64 a,i64 b) {
    if(a<0||b<1) throw std::invalid_argument("nonnegative fraction");
    i64 q=b/std::gcd(a,b); int u=0,v=0;
    while(q%2==0) {q/=2;++u;} while(q%5==0) {q/=5;++v;}
    return {digits10(a/b),std::max(u,v),q==1?0:order_mod(10,q)};
}
} // namespace finals_math
```

1.6 Approximation as a construction

1.6.1 Fuzzy Factorization (2025 F)

Allowed relative error 10^{-9} changes the task. If X has more than ten digits, keep ten leading digits m and discard e places:

$$X = m10^e + r, \quad 0 \le r < 10^e, \quad m \ge 10^9.$$

Choose $Y = m10^e$. Then

$$\frac{|X-Y|}{X} < \frac{1}{m} \le 10^{-9}.$$

Factor only $m < 10^{10}$; add e to exponents of 2 and 5. Merge repeated primes. For at most ten digits choose $Y = X$. Time $O(|X| + \sqrt{m})$; no huge integer arithmetic.

Example $X = 123456789012345$: $m = 1234567890$, $e = 5$, $m = 2 \cdot 3^2 \cdot 5 \cdot 3607 \cdot 3803$. Thus $Y = 2^6 3^2 5^6 \cdot 3607 \cdot 3803$. Discarded remainder 12345 obeys $12345 \cdot 10^9 \le X$. Tolerance 10^{-k} similarly permits $k+1$ leading digits, but verify factorization cost and output prime limits. Empty factor map means one; adapt the required output format. Positive input, no leading zeros.

FuzzyFactorization.h
Description: Fuzzy factorization construction.
"Prelude.h", "FactorSieve.h" bd458d, 13 lines

```
namespace finals_math {
// Positive canonical decimal input (no leading zero); factor the ten–digit prefix,
// then account for trailing powers of ten. Empty factors means Y=1.
inline std::map<i64,int> fuzzy_factors(const std::string& s) {
    if(s.empty()||s[0]=='0')throw std::invalid_argument("positive canonical decimal");
    for(char c:s)if(c<'0'||c>'9')throw std::invalid_argument("decimal digits");
    i64 m=0; int keep=int(std::min<std::size_t>(10,s.size()));
    for(int i=0;i<keep;++i)m=m*10+s[i]-'0';
    int e=int(s.size())-keep; std::map<i64,int> f;
    for(auto [p,k]:factor(m))f[p]=k;
    if(e) {f[2]+=e;f[5]+=e;} return f;
}
} // namespace finals_math
```

1.7 Combinatorics

1.7.1 Choose the sample objects

A choices each followed by exactly B choices give AB. If counts vary, sum them. Disjoint cases add; overlaps need correction. Ordered distinct k-tuples: $n!/(n-k)!$; unordered subsets: $\binom{n}{k} = n!/(k!(n-k)!)$; ordered repetition: n^k. Five people: president/secretary 20, two-person committee 10. Stars and bars: k ordered bins, total n identical items; nonempty $\binom{n-1}{k-1}$, empty allowed $\binom{n+k-1}{k-1}$. For $k \ge 1$; upper capacities need additional reasoning.

Pascal: $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$, edge values one. Works modulo any positive modulus: $O(n^2)$ time, $O(n)$ memory; update a single row in decreasing k. Prime factorial tables: $O(N + \log p)$ preprocessing, $O(1)$ queries, $O(N)$ memory. Require prime p and $N < p$. At $N \ge p$, $N!$ is zero modulo p; composite moduli may fail earlier. The constructor trusts primality. No automatic Lucas/composite support.

Combinations.h
Description: Combinations under a prime modulus.
"Prelude.h", "ModularArithmetic.h" 288edc, 18 lines

```
namespace finals_math {
// Prime p is a CALLER precondition; n<p. Constructor does not test primality.
struct PrimeCombinations {
    i64 p; std::vector<i64> fac,invfac;
    PrimeCombinations(int n,i64 prime):p(prime) {
        if(n<0||p<=1||n>=p)throw std::invalid_argument("need 0<=n<prime");
        fac.resize(n+1,1); invfac.resize(n+1,1);
        for(int i=1;i<=n;++i)fac[i]=mul_mod(fac[i-1],i,p);
        invfac[n]=pow_mod(fac[n],p-2,p);
        for(int i=n;i>0;--i)invfac[i-1]=mul_mod(invfac[i],i,p);
    }
    i64 choose(int n,int k)const {
        if(n<0||n>=int(fac.size()))throw std::invalid_argument("choose range");
        if(k<0||k>n)return 0;
        return mul_mod(fac[n],mul_mod(invfac[k],invfac[n-k],p),p);
    }
};
} // namespace finals_math
```

1.7.2 Contributions and complements

Zero-based index i appears in $(i+1)(n-i)$ subarrays, so

$$\sum_{\text{subarrays}} \sum a_i = \sum_i a_i (i+1)(n-i).$$

Example $[2,1,3]$: $6 + 4 + 9 = 19$. Widen all products. For unordered pairs:

$$\sum_{i<j} (a_i + a_j) = (n-1) \sum_i a_i,$$

$$\sum_{i<j} (a_i - a_j)^2 = n \sum_i a_i^2 - (\sum_i a_i)^2.$$

At least one good feature = total minus zero good features. A maximal bad run of length L contributes $L(L+1)/2$ all-bad substrings. Circular input needs merged boundary runs and a separate all-bad case; the linear formula alone is insufficient.

1.7.3 Inversions and repeated copies

Inversion: $i < j$, $a_i > a_j$ strictly. Equal values do not count. Fenwick scan adds previously seen minus previous values $\le a_i$. Coordinate compression preserves comparisons; $O(n \log n)$ time/space $O(n)$. For K copies of S:

$$\text{inv}(S^K) = K \text{ within} + \binom{K}{2} \text{ cross},$$

where cross $= \#\{(u,v) : S_u > S_v\}$ without an index-order restriction inside one copy. Every earlier-copy position precedes every later-copy position. For $S = ba$, $K = 3$, answer $3 + 3 = 6$. This is the decomposition in 2023 I. Use wide/modular products.

Inversions.h
Description: Strict inversions by compressed Fenwick tree.
"Prelude.h", "IntegerDivision.h" fed933, 16 lines

```
namespace finals_math {
// Generic comparisons: counts i<j with a[i]>a[j], O(n log n), preserves input.
inline i64 inversions(std::vector<i64> a) {
    auto vals=a;
    std::sort(vals.begin(),vals.end());
    vals.erase(std::unique(vals.begin(),vals.end()),vals.end());
    std::vector<i64> bit(vals.size()+1); i64 answer=0,seen=0;
    for(i64 x:a) {
        int k=int(std::lower_bound(vals.begin(),vals.end(),x)-vals.begin())+1;
        i64 le=0; for(int j=k;j;j-=j&-j)le+=bit[j];
        answer=narrow(i128(answer)+seen-le); ++seen;
        for(int j=k;j<int(bit.size());j+=j&-j)++bit[j];
    }
    return answer;
}
} // namespace finals_math
```

1.8 Probability and expectation

Uniform outcomes must be specified: ordered occurrence pairs with replacement number N^2, without replacement $N(N-1)$. Equal values may still be different occurrences.

$$\mathbb{E}[X] = \sum_x x \Pr(X = x), \qquad \mathbb{E}[\sum I_j] = \sum \Pr(I_j = 1).$$

Linearity needs no independence. Multiplying probabilities does: $\Pr(A \cap B) = \Pr(A)\Pr(B \mid A)$. Random permutation expected inversions: each pair has probability $1/2$, so $n(n-1)/4$. Expectation need not be a possible outcome.

Self loop: $E = c + qE + \sum_j p_j E_j$ gives $E = (c + \sum_j p_j E_j)/(1-q)$ for $q < 1$. A permanent positive-cost self loop has infinite expectation. Modulo prime p, fraction A/B means AB^{-1} only if $p \nmid B$; reduce exact cancellations first if needed. A real finite expectation need not have an invertible modular denominator. Never modularize a decimal approximation. $\mathbb{E}[X^2] \neq \mathbb{E}[X]^2$ generally; $\mathrm{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$.

1.9 Proof and inequality checks

An invariant needs initial truth and preservation by every operation. Try sum, parity, weighted sum, gcd, multiset, order. Invariant equality is necessary; prove sufficiency by a construction. A decreasing nonnegative integer proves termination. Binary search needs a monotone predicate, not just a fast checker. Greedy needs an exchange preserving feasibility and not worsening cost. Brute-force tiny cases find counterexamples; passing them is not proof.

Useful sums: $\sum_{i=1}^{n} i = n(n+1)/2$, $\sum_{i=0}^{k} r^i = (r^{k+1}-1)/(r-1)$ for $r \neq 1$. Before modular division, cancel an exact factor or verify invertibility. For error bounds compare with the original denominator; check strict vs nonstrict inequalities and small inputs separately.

1.10 Integer geometry

For vectors u, v, $u \cdot v = u_x v_x + u_y v_y$, $u \times v = u_x v_y - u_y v_x$. Dot zero: perpendicular (nonzero vectors); positive/negative: acute/obtuse. $\mathrm{orient}(A,B,C) = (B-A) \times (C-A)$: positive left, negative right, zero collinear. Compare squared distances; no square root needed. Twice triangle area is $|\,\mathrm{orient}\,|$.

Point on segment: orientation zero and both coordinates within inclusive endpoint ranges. Degenerate $A = B$ works. Segment intersection: opposite orientation signs for both lines, plus on-segment zero cases; collinear segments reduce to interval overlap. Compare signs, do not multiply cross products. Polygon in boundary order: $2A = |\sum_i P_i \times P_{i+1}|$ for a simple polygon. Self intersections give algebraic area; sorting by x is not boundary order.

Helpers use 128-bit operations and require coordinate magnitudes $\leq 10^{18}$. Arbitrary full-range 64-bit coordinates can overflow even 128 bits; polygon accumulation needs a separate bound in n. Integer predicates are exact; real intersections/distances need tolerances based on the statement, not a universal EPS.

IntegerGeometry.h
Description: Exact integer geometry.
`"Prelude.h"` 45bdbc, 9 lines

```
namespace finals_math {
struct Point { i64 x,y; };
// Coordinate magnitudes <=1e18; products/sums fit signed 128
     bits under this bound.
inline i128 orient(Point a,Point b,Point c) {
    return (i128(b.x)-a.x)*(i128(c.y)-a.y)-(i128(b.y)-a.y)*(
        i128(c.x)-a.x);
}
inline i128 dot(Point a,Point b) {return i128(a.x)*b.x+i128(a.y
    )*b.y;}
inline i128 dist2(Point a,Point b) {i128 x=i128(a.x)-b.x,y=i128
    (a.y)-b.y;return x*x+y*y;}
} // namespace finals_math
```